

APOSTILA PRÁTICA DE GIT & GITHUB

Versionamento de Código, Fluxos de Trabalho Colaborativos e Construção de Portfólio Profissional para Estudantes de TI

Sobre esta Apostila

Esta apostila foi projetada para ser o ponto de partida ideal para qualquer estudante ingressando na área de Tecnologia da Informação (TI). Independentemente de sua especialidade — seja desenvolvimento de software, ciência de dados, engenharia de redes, segurança da informação ou gestão de produtos de TI — o domínio do controle de versão e do uso da plataforma GitHub é uma das competências mais exigidas pelo mercado de trabalho atual. Aqui, você aprenderá desde os conceitos mais básicos até fluxos práticos de trabalho e o uso de IA para maximizar sua produtividade.

Informação do Documento	Detalhes Acadêmicos
Público-Alvo	Estudantes da Área de TI (Sistemas, Ciência da Computação, Engenharia, Análise de Sistemas)
Nível Requerido	Iniciante (Do Absoluto Zero)
Garantia de Qualidade	Versão 2.0 (Revisada, Integrada com CLI, IA e Portfólio de Carreira)
Escopo Tecnológico	Git Web GUI (Interface Gráfica), Git CLI (Linha de Comando) e GitHub Copilot

1. O Ecossistema de Controle de Versão

O controle de versão é um pilar vital em projetos de tecnologia. Ele permite registrar o histórico completo de alterações em arquivos, analisar e comparar mudanças ao longo do tempo e reverter modificações caso ocorram falhas inesperadas. Em equipes multidisciplinares de TI, essa tecnologia evita que o trabalho de diferentes profissionais seja sobrescrito, mitigando drasticamente os riscos de erros operacionais.

1.1 Git: O Motor do Rastreamento

Criado em 2005 por **Linus Torvalds** (o mesmo engenheiro que desenvolveu o kernel do sistema operacional Linux), o **Git** é um software livre de controle de versões distribuído. O seu principal propósito é registrar todas as modificações realizadas nos arquivos do código-fonte e coordenar o trabalho cooperativo de equipes de desenvolvimento em um repositório compartilhado. Pesquisas globais da comunidade, como a do Stack Overflow, indicam que **mais de 90% dos profissionais de tecnologia** utilizam o Git ativamente no dia a dia.

1.2 GitHub: A Nuvem e a Rede Social da Tecnologia

O **GitHub** é um serviço de hospedagem de código-fonte baseado na web, fundado em fevereiro de 2008 por Chris Wanstrath, PJ Hyett, Tom Preston-Werner e Scott Chacon utilizando a estrutura do Ruby on Rails. Diferente do Git (que roda localmente no seu computador), o GitHub opera na nuvem e adiciona recursos de colaboração social, discussões em fóruns e controle de equipe. Em outubro de 2018, a plataforma foi adquirida pela **Microsoft** por US\$ 7,5 bilhões sob a liderança do CEO Satya Nadella, que na ocasião reafirmou o compromisso da corporação em capacitar desenvolvedores através da inovação e da liberdade de código aberto.

FATO HISTÓRICO RELEVANTE: O GitHub abriga atualmente mais de 100 milhões de desenvolvedores ativos no mundo e mais de 330 milhões de repositórios. Grandes corporações mundiais como Google, Microsoft e a fundação do WordPress utilizam a plataforma para gerenciar, desenvolver e integrar seus ecossistemas tecnológicos em larga escala.

Resumo Prático: Git vs. GitHub

Característica	Git	GitHub
Natureza	Software executado localmente na máquina do usuário.	Serviço de hospedagem em nuvem acessível via web.
Propósito Principal	Registrar alterações em arquivos e manter o histórico do código.	Compartilhar repositórios Git, colaborar e integrar equipes.
Interface	Linha de Comando (CLI) ou ferramentas gráficas locais.	Interface Web gráfica e ferramentas interativas baseadas na nuvem.
Internet	Funciona de forma offline.	Requer conexão à internet para sincronização remota.

2. O GitHub em Diferentes Áreas de Tecnologia

Muitos estudantes acreditam erroneamente que o GitHub serve apenas para programadores focados em desenvolvimento de software tradicional. A realidade do mercado moderno de TI é que a plataforma se expandiu para abranger virtualmente todas as carreiras técnicas de tecnologia. Seja na criação de documentações técnicas, governança de dados ou automação de servidores, o controle de versão centralizado é uma necessidade onipresente.

Carreira em TI	Como Utiliza o GitHub	Principais Recursos Empregados
Dev de Software	Hospedagem de código em equipe, revisão de código (Code Review), testes integrados e fusão de ramificações paralelas.	Branches, Pull Requests, Commits, Merges, CLI.
Ciência de Dados	Versionamento de scripts matemáticos, compartilhamento de Jupyter Notebooks e análise de estatísticas corporativas.	GitHub Repositories, Python scripts, versionamento de planilhas.
DevOps & Infraestrutura	Gerenciamento de infraestrutura como código (IaC), configuração automatizada de servidores VPS e automação de deploy.	Integrações de CI/CD, conexões via SSH, repositórios privados.
UI/UX & Gestão de TI	Versionamento e feedback de arquivos de design, documentação técnica cooperativa e planejamento ágil de projetos.	Project Boards, Issues, arquivo README.md, Markdown.

3. Primeiros Passos Práticos via Interface Web

Para acelerar o aprendizado inicial e reduzir a barreira técnica, iniciaremos o uso do GitHub diretamente pela sua interface web. Isso permite compreender os fluxos de trabalho fundamentais de forma visual antes de partirmos para comandos avançados do terminal de comando.

Passo 1: Criando Sua Conta Gratuita

1. Acesse o site oficial em **github.com**.
2. No canto superior direito, clique em **'Sign up'** (Inscrever-se).
3. Escolha um nome de usuário único (ele será seu link de identidade na TI), forneça seu e-mail ativo e crie uma senha segura.
4. Confirme seu cadastro seguindo o link de validação enviado ao seu e-mail.
5. O GitHub oferecerá planos diferentes: selecione o **Plano Gratuito**, que atende perfeitamente aos estudantes, disponibilizando 500 MB de armazenamento em nuvem e acesso ilimitado a repositórios públicos e privados (com limite de colaboração compartilhada para até 3 usuários simultâneos).

Passo 2: Criando o Seu Primeiro Repositório

O repositório é uma pasta virtual na nuvem onde todos os arquivos de um projeto ficam concentrados de forma organizada. Siga as etapas para criar o seu:

1. Faça login na sua conta do GitHub.
2. No menu superior direito, clique no símbolo de mais (+) e selecione **'New repository'** (Novo repositório).
3. Preencha o **Repository name** (escolha um nome curto e separado por traço, ex: *meu-primeiro-projeto*).
4. Escolha a visibilidade: **Public** (seu código fica visível para o mundo, ótimo para portfólio acadêmico) ou **Private** (acesso exclusivo seu e de pessoas autorizadas).
5. **EXTREMAMENTE IMPORTANTE:** Marque a opção **'Add a README file'**. Este arquivo será criado no formato Markdown (.md) e funcionará como a 'capa' do seu repositório, onde você explicará detalhadamente o objetivo do projeto.

4. O Fluxo de Trabalho (Workflow) Colaborativo

Para trabalharmos em equipe de forma eficiente sem corromper o projeto principal, o GitHub utiliza um fluxo padrão de ramificações, commits e solicitações de mesclagem. A seguir, abordaremos este fluxo simulando uma alteração no seu projeto recém-criado.

Passo 3: Criando uma Ramificação (Branch)

Por padrão, todo repositório possui uma branch principal chamada **Main**, que armazena a versão estável e em produção do seu projeto. Para testar recursos ou fazer alterações experimentais sem afetar a Main, você deve criar uma branch separada:

1. Acesse o seu repositório e clique no botão que indica a branch ativa (geralmente escrito '**main**').
2. No campo de busca que se abre, digite o nome de sua nova ramificação (ex: *alteracoes-iniciais*).
3. Clique em '**Create branch**'. Agora você está posicionado em uma cópia paralela e segura do projeto.

Passo 4: Salvando Alterações (Commits)

Um **Commit** funciona como um ponto de salvamento na linha do tempo do seu projeto (uma 'foto' que preserva o estado dos arquivos naquele instante):

1. Garanta que você está na branch que acabou de criar (*alteracoes-iniciais*).
2. Abra o arquivo *README.md* e clique no ícone de lápis para começar a editar diretamente no navegador.
3. Digite alguma alteração textual simples.
4. No final da página, vá em '**Commit changes...**'. Você deve digitar uma **mensagem de commit descritiva** explicando de forma objetiva o que foi alterado. Por fim, confirme clicando no botão para salvar.

Passo 5: Propondo Mudanças via Pull Request (PR)

O **Pull Request** é uma solicitação para integrar as alterações feitas em sua branch paralela de volta ao ramo principal (Main). É a ferramenta mais crucial de colaboração, permitindo que outros profissionais analisem, comentem e aproveem seu trabalho:

1. O GitHub exibirá um banner sugerindo a abertura do PR. Caso contrário, vá até a aba '**Pull requests**' e clique em '**New pull request**'.
2. No menu de comparação, verifique se a branch de origem (*alteracoes-iniciais*) está apontando para a branch de destino (*main*).
3. A plataforma exibirá as diferenças entre os códigos de forma visualizada em cores: **linhas excluídas em vermelho** e **linhas adicionadas em verde**.
4. Preencha um título descritivo para seu PR, insira notas explicativas e clique em '**Create pull request**'.

Passo 6: Incorporando as Mudanças (Merge)

Se as alterações foram aprovadas na revisão do time, elas podem ser incorporadas à branch de produção:

1. Na tela do próprio Pull Request criado, role a página até o botão verde '**Merge pull request**' e clique nele.
2. Confirme em '**Confirm merge**'. O status do Pull Request mudará para roxo (*Merged*), indicando sucesso.
3. Agora que as alterações fazem parte da Main, você pode excluir a branch temporária com segurança clicando no botão '**Delete branch**'.

COLABORAÇÃO ACADÊMICA — TRABALHOS EM EQUIPE: No ambiente universitário de TI, os alunos podem criar **Forks** para bifurcar repositórios de seus professores ou colegas, gerando cópias independentes em suas contas para treinar modificações livres. Além disso, a aba de **Issues** (Problemas) deve ser usada ativamente para rastrear bugs, tarefas e metas dos trabalhos acadêmicos de forma organizada.

5. Da Web para o Computador: Comandos via Terminal (CLI)

Embora a interface gráfica seja intuitiva, o trabalho profissional em TI exige o domínio da Interface de Linha de Comando (CLI). Abaixo, estão detalhados os comandos elementares do Git que você usará no terminal do seu computador para manipular projetos locais e sincronizá-los com a nuvem do GitHub.

Comandos Fundamentais de Sincronização Local-Nuvem

A. Clonar um Repositório: Faz o download de uma cópia completa de um repositório remoto do GitHub diretamente para o disco rígido do seu computador.

```
git clone https://github.com/usuario/nome-do-repositorio.git
```

B. Adicionar Alterações: Adiciona as modificações realizadas nos seus arquivos locais para a 'área de preparação' (staging area), preparando-as para o commit.

```
git add nome_do_arquivo.py    # Para adicionar um arquivo específico
git add .                     # Para adicionar TODAS as alterações locais
```

C. Registrar Alterações (Commit): Salva localmente as alterações adicionadas à área de preparação com uma mensagem explicando as modificações.

```
git commit -m "Adiciona nova funcionalidade de cálculo matemático"
```

D. Enviar para a Nuvem (Push): Transmite todas as suas alterações confirmadas localmente de volta para o repositório remoto hospedado no GitHub.

```
git push origin nome-da-branch
```

E. Sincronizar Código Local (Pull): Atualiza o seu computador local com as mudanças mais recentes enviadas por outros colegas diretamente na nuvem do GitHub.

```
git pull origin main
```

6. Inteligência Artificial no Desenvolvimento: GitHub Copilot

O **GitHub Copilot** é uma ferramenta revolucionária de inteligência artificial desenvolvida pela OpenAI em parceria com o GitHub e a Microsoft. Baseado em modelos avançados de Aprendizado de Máquina (Machine Learning) treinados em bases massivas de dados públicos, o Copilot atua como um assistente em tempo real dentro do seu ambiente de desenvolvimento (IDE) como Visual Studio Code, JetBrains ou Visual Studio.

Ele fornece sugestões automatizadas de blocos inteiros de código à medida que você escreve. Você pode interagir com a IA escrevendo comentários explicativos em linguagem natural (Português, Inglês, etc.) e o Copilot gerará o algoritmo correspondente de forma instantânea. Se você for um **estudante acadêmico verificado** no GitHub Education, o uso do Copilot pode ser liberado de forma totalmente gratuita pelo GitHub.

```
# Exemplo em Python: Comentário em Linguagem Natural
# Função para calcular a soma de dois números e verificar se o resultado é par
def verificar_soma_par(a, b):
    soma = a + b
    eh_par = (soma % 2 == 0)
    return soma, eh_par

# O Copilot sugere e completa automaticamente todo o trecho acima com apenas um toque na
tecla TAB!
```


7. O GitHub como seu Portfólio Profissional de TI

Atualmente, no mercado corporativo de TI, o seu perfil do GitHub funciona como seu verdadeiro currículo técnico e principal cartão de visitas. Recrutadores e equipes de engenharia avaliam ativamente o seu perfil para verificar o nível técnico das suas contribuições, a qualidade de organização do seu código, seu histórico de consistência em projetos e sua capacidade de colaborar.

Estratégias Fundamentais de Vitrine Profissional

- **Mantenha seus Repositórios Organizados:** Crie repositórios claros para os projetos da faculdade, documentando o que o projeto faz no arquivo README.md.
- **Perfil Pessoal (README):** Você pode criar um repositório especial com o mesmo nome do seu usuário (ex: *github.com/seu-usuario/seu-usuario*). Ao inicializar este repositório com um arquivo README.md, ele se tornará o painel central do seu perfil, permitindo adicionar apresentações, suas competências técnicas e contatos profissionais.
- **Participe de Projetos de Código Aberto (Open Source):** Contribuir para projetos públicos e interagir com comunidades de desenvolvedores é a melhor forma de amadurecer suas habilidades técnicas na prática e ganhar notoriedade global.

8. Desafio Prático de Laboratório: Seu Primeiro Projeto de TI

Chegou a hora de aplicar todo o conhecimento acumulado nesta apostila! Siga o passo a passo a seguir para realizar o fluxo de trabalho completo utilizado diariamente pelas grandes corporações globais de tecnologia.

Etapa A: Configurando o Projeto

1. Faça login no GitHub e crie um novo repositório público chamado **desafio-ti-github**.
2. Lembre-se de marcar a caixa de seleção **'Add a README file'** antes de confirmar o salvamento do repositório.

Etapa B: Criando a Branch Experimental

1. Na tela inicial do repositório, clique na lista suspensa que indica a branch **main**.
2. Digite o nome **perfil-profissional** no campo de pesquisa e clique na opção para criar essa branch paralela.

Etapa C: Editando o Arquivo README.md

1. Com a branch *perfil-profissional* ativa, abra o arquivo *README.md* e clique no ícone de edição (lápiz).
2. Edite o arquivo substituindo o conteúdo original pelo seguinte modelo de apresentação profissional e salve com um commit descritivo:

```
# Olá, meu nome é [Seu Nome]!  
Estudante ingressante na área de Tecnologia da Informação (TI).  
  
## Meus Principais Objetivos de Carreira:  
1. Aprender desenvolvimento de software colaborativo com foco em código limpo.  
2. Compreender a lógica de dados e automação de processos na nuvem.  
  
## Minhas Habilidades em Desenvolvimento:  
- Linguagens básicas em estudo: Python, Markdown e lógica de programação.  
- Ferramentas: Git, GitHub e editores de código (IDEs).
```

Etapas D: Integrando as Alterações (Merge)

1. Vá até a aba **Pull requests** e clique no botão verde para iniciar uma solicitação.
2. Selecione a branch *perfil-profissional* para fundir na branch *main*. Observe o comparativo de cores das modificações e crie o Pull Request.
3. Sendo o administrador do repositório, clique no botão **Merge pull request** e confirme em **Confirm merge**.
4. Exclua a branch *perfil-profissional* clicando no botão de limpeza após o merge com sucesso.

Parabéns! Você acaba de estruturar a vitrine do seu portfólio profissional de TI na plataforma e dominou o fluxo de versionamento corporativo!